

Dominando las CTE en SQL

Guía de referencia teórica y resolución de ejercicios prácticos sobre Common Table Expressions y Subconsultas.

¿Qué son las Common Table Expressions (CTE)?

Una **CTE** es un conjunto de resultados temporal con nombre que se define dentro del ámbito de ejecución de una única instrucción `SELECT`, `INSERT`, `UPDATE` o `DELETE`. Se introducen mediante la cláusula `WITH`. Su principal ventaja frente a las subconsultas derivadas es que mejoran drásticamente la legibilidad del código (permitiendo leer de arriba hacia abajo) y pueden ser referenciadas múltiples veces en la misma consulta.

1. CTE Simple: Consiste en un único bloque de consulta temporal. Es ideal para aislar lógica de negocio (por ejemplo, filtrar qué registros son válidos) antes de realizar agrupaciones complejas o cruces en la consulta principal.

2. CTE Encadenadas (Pipeline): Permiten definir múltiples CTE separadas por comas. El resultado de la primera puede ser consultado por la segunda, y así sucesivamente. Este patrón de "pipeline" es excelente para transformar datos por fases: *Limpiar* → *Enriquecer* → *Agregar*, facilitando enormemente la depuración de consultas analíticas pesadas.

3. CTE Recursivas: Son la única estructura en SQL nativo capaz de iterar sobre sí misma. Se componen de dos partes unidas por un `UNION ALL`:

- **El miembro ancla:** Una consulta base que devuelve los primeros registros (nivel 0 o raíz) y se ejecuta solo una vez.
- **El miembro recursivo:** Una consulta que se referencia a la propia CTE, obteniendo los "hijos" del nivel anterior. Se ejecuta repetidamente hasta que no devuelve más filas.

Cómo identificar lo que pide el enunciado

Traducir un requerimiento de negocio a código SQL requiere aplicar un proceso analítico paso a paso. Antes de escribir `SELECT`, evalúa el enunciado respondiendo a estas preguntas:

1. ¿Cuál es el nivel de granularidad final?

Busca palabras como "por género", "para cada usuario", "totales". Esto define tu `GROUP BY`. Si te piden el total general, no hay `GROUP BY`.

2. ¿Qué datos deben excluirse previamente?

Frases como "solo reproducciones válidas" o "al menos 30 segundos" indican filtros de la cláusula `WHERE`. Estos deben aplicarse antes de cualquier agrupación.

3. ¿El filtro depende de un cálculo agregado?

Si el enunciado dice "usuarios que tengan **más de 5** reproducciones", estás filtrando el resultado de una métrica. Eso pertenece a la cláusula `HAVING`, no al `WHERE`.

4. ¿Se requieren datos que no quieres mostrar pero condicionan el resultado?

Palabras como "que nunca han...", "que existan en...", "al menos un..." sugieren el uso de subconsultas lógicas como `EXISTS`, `NOT EXISTS` o `IN`, en lugar de usar `JOIN`, lo que evita la duplicación de filas.

Ejercicios de Refuerzo · Parte 1

Ejercicio 1

Enunciado: El sello de Luna Roja quiere saber qué canciones del catálogo duran más que la canción más larga de Luna Roja. Resuélvelo con una subconsulta escalar que obtenga esa duración máxima a partir del nombre del artista. Ordena de mayor a menor duración.

```
SELECT titulo, duracion_seg FROM dbo.canciones WHERE duracion_seg > ( SELECT
MAX(c.duracion_seg) FROM dbo.canciones AS c JOIN dbo.artistas AS a ON c.artista_id =
a.artista_id WHERE a.nombre = N'Luna Roja' ) ORDER BY duracion_seg DESC;
```

Explicación: La subconsulta en el `WHERE` está garantizada a devolver un único valor (el máximo), convirtiéndose en una subconsulta escalar. Al evaluarse primero, la consulta principal simplemente compara la duración de todas las canciones contra ese número estático calculado.

titulo	duracion_seg
Fábrica 4AM	412
Marea Alta	356
Latido	330
Forja	305
Corriente Alterna	234
Rimas de Barrio	214

Ejercicio 2

Enunciado: El equipo de producto está diseñando una interfaz. Obtén, con una subconsulta de lista (IN), los títulos de las canciones que se han reproducido alguna vez desde un Smart TV. Ordena alfabéticamente.

```
SELECT titulo FROM dbo.canciones WHERE cancion_id IN ( SELECT cancion_id FROM
dbo.reproducciones WHERE dispositivo = N'Smart TV' ) ORDER BY titulo;
```

Explicación: El operador `IN` compara cada `cancion\_id` con la lista de IDs devuelta por la subconsulta. Usar `IN` en lugar de un `JOIN` evita registros duplicados si una misma canción se ha reproducido múltiples veces desde un Smart TV.

titulo
Diamantes Rotos
Galaxia Dembow
Neón
Verano en Bucle

Ejercicio 3

Enunciado: El equipo de catálogo necesita detectar artistas fichados que todavía no tienen ninguna canción publicada. Resuélvelo con NOT EXISTS.

```
SELECT nombre FROM dbo.artistas AS a WHERE NOT EXISTS ( SELECT 1 FROM
dbo.canciones AS c WHERE c.artista_id = a.artista_id );
```

Explicación: `NOT EXISTS` evalúa fila por fila (subconsulta correlacionada). Por cada artista, busca si hay al menos un registro en `canciones` que coincida con su ID. Si la subconsulta no devuelve nada (no tiene canciones), la condición es verdadera y el artista se incluye en el resultado.

nombre
Sara Cometa

Ejercicio 4

Enunciado: El equipo comercial quiere ofrecer un descuento en Premium a los usuarios que han escuchado al menos un anuncio. Obtén su nombre y plan con EXISTS. Ordena por nombre.

```
SELECT nombre_usuario, plan_suscripcion FROM dbo.usuarios AS u WHERE EXISTS (
SELECT 1 FROM dbo.reproducciones AS r WHERE r.usuario_id = u.usuario_id AND
r.tipo_contenido = N'Anuncio' ) ORDER BY nombre_usuario;
```

Explicación: De manera similar al ejercicio anterior, `EXISTS` es la estructura óptima para comprobaciones de existencia. A diferencia de un `JOIN` con `DISTINCT`, `EXISTS` detiene la búsqueda del motor de base de datos en cuanto encuentra la primera coincidencia (primer anuncio), mejorando el rendimiento.

nombre_usuario	plan_suscripcion
dani_dj	Free
marta_rock	Free
nachox	Free

Ejercicio 5

Enunciado: Obtén usuarios con 4 o más reproducciones válidas. Resuélvelo con tabla derivada y luego reescríbelo con CTE. Ordena por reproducciones y usuario.

```
-- 1. Con Tabla Derivada SELECT u.nombre_usuario, t.reproducciones_validas FROM (
SELECT usuario_id, COUNT(*) AS reproducciones_validas FROM dbo.reproducciones WHERE
tipo_contenido = N'Canción' AND segundos_escuchados >= 30 GROUP BY usuario_id ) AS t JOIN
dbo.usuarios AS u ON u.usuario_id = t.usuario_id WHERE t.reproducciones_validas >= 4
ORDER BY t.reproducciones_validas DESC, u.nombre_usuario; -- 2. Con CTE (Solución
preferida) WITH validas_agrupadas AS ( SELECT usuario_id, COUNT(*) AS
reproducciones_validas FROM dbo.reproducciones WHERE tipo_contenido = N'Canción' AND
segundos_escuchados >= 30 GROUP BY usuario_id ) SELECT u.nombre_usuario,
v.reproducciones_validas FROM validas_agrupadas AS v JOIN dbo.usuarios AS u ON
u.usuario_id = v.usuario_id WHERE v.reproducciones_validas >= 4 ORDER BY
```

Ejercicio 6

Enunciado: Para canciones de Nébula y DJ Coral, muestra título y total de reproducciones (válidas o no), incluidas las que tienen cero. Usa subconsulta correlacionada en el SELECT y una subconsulta de lista (IN) para artistas.

```
SELECT c.titulo, (SELECT COUNT(*) FROM dbo.reproducciones AS r WHERE r.cancion_id = c.cancion_id) AS total_reproducciones FROM dbo.canciones AS c WHERE c.artista_id IN (SELECT artista_id FROM dbo.artistas WHERE nombre IN (N'Nébula', N'DJ Coral')) ORDER BY total_reproducciones DESC, c.titulo;
```

Explicación: Colocar una subconsulta en el `SELECT` obliga a que se evalúe por cada fila devuelta por la tabla principal (`canciones`). Dado que cuenta las apariciones en `reproducciones`, si no hay coincidencias la función `COUNT(\*)` devuelve `0`, resolviendo el problema de las canciones sin reproducciones de forma natural sin requerir un `LEFT JOIN`.

titulo	total_reproducciones
Diamantes Rotos	2
Humo Violeta	2
Marea Alta	2
Sin Cobertura	1
Latido	0

Ejercicio 7

Enunciado: Un compañero escribió una consulta con NOT IN para encontrar canciones que ningún usuario Free ha escuchado, pero devuelve 0 filas. Explica por qué falla y reescríbela correctamente con NOT EXISTS.

```
-- Código original fallido: -- SELECT titulo FROM dbo.canciones WHERE cancion_id NOT IN ( -- SELECT r.cancion_id FROM dbo.reproducciones r JOIN dbo.usuarios u... -- ) --  
Solución correcta: SELECT c.titulo FROM dbo.canciones AS c WHERE NOT EXISTS ( SELECT 1 FROM dbo.reproducciones AS r JOIN dbo.usuarios AS u ON u.usuario_id = r.usuario_id WHERE r.cancion_id = c.cancion_id AND u.plan_suscripcion = N'Free' ) ORDER BY c.titulo;
```

Explicación: El operador `NOT IN` presenta una falla crítica frente a valores `NULL`. Si la subconsulta interna devuelve tan solo un registro donde `cancion\_id` es `NULL` (por ejemplo, reproducciones huérfanas o errores de integridad), la condición entera se evalúa como UNKNOWN por la lógica trivaluada de SQL, devolviendo cero resultados en total. `NOT EXISTS` no compara valores directamente, evalúa si un conjunto está vacío, siendo inmune a este problema.

titulo
Diamantes Rotos
Galaxia Dembow
Humo Violeta
Latido
Neón
Sin Cobertura

Ejercicio 8

Enunciado: Identifica usuarios del *staging* que aún no están cargados en la tabla final. Después, convierte la consulta en un INSERT ... SELECT idempotente y comprueba que al ejecutarse dos veces no duplica filas.

```
-- 1. Consulta de identificación: SELECT usuario_id, nombre_usuario,
plan_suscripcion FROM dbo.stg_usuarios AS s WHERE NOT EXISTS ( SELECT 1 FROM dbo.usuarios
AS u WHERE u.usuario_id = s.usuario_id ); -- 2. Operación Idempotente (INSERT ...
SELECT): INSERT INTO dbo.usuarios (usuario_id, nombre_usuario, plan_suscripcion) SELECT
s.usuario_id, s.nombre_usuario, s.plan_suscripcion FROM dbo.stg_usuarios AS s WHERE NOT
EXISTS ( SELECT 1 FROM dbo.usuarios AS u WHERE u.usuario_id = s.usuario_id );
```

Explicación: Una operación "idempotente" es aquella que puede ejecutarse múltiples veces garantizando el mismo estado final en la base de datos. Gracias al `NOT EXISTS`, la primera ejecución detectará e insertará los nuevos IDs. En una ejecución posterior, `NOT EXISTS` filtrará todos los registros (ya fueron ingresados), logrando así la salida (*0 rows affected*) y evitando violaciones de clave primaria.

Ejercicio 9

Enunciado: Pipeline de tres CTE encadenadas (limpiar → enriquecer → agregar) para calcular minutos de música válida por país y plan de suscripción.

```
WITH limpias AS ( SELECT usuario_id, segundos_escuchados FROM dbo.reproducciones
WHERE tipo_contenido = N'Canción' AND segundos_escuchados >= 30 ), enriquecidas AS (
SELECT l.segundos_escuchados, u.pais, u.plan_suscripcion FROM limpias AS l JOIN
dbo.usuarios AS u ON u.usuario_id = l.usuario_id ), agregadas AS ( SELECT pais,
plan_suscripcion, COUNT(*) AS reproducciones, CAST(SUM(segundos_escuchados) / 60.0 AS
decimal(6,1)) AS minutos FROM enriquecidas GROUP BY pais, plan_suscripcion ) SELECT pais,
plan_suscripcion, reproducciones, minutos FROM agregadas ORDER BY minutos DESC;
```

Explicación: Dividir la consulta por fases aísla las responsabilidades lógicas. El factor `60.0` (forzando decimal) evita el redondeo a entero que ocurre en SQL Server si ambos operandos son enteros.

pais	plan_suscripcion	reproducciones	minutos
ES	Premium	10	29.4
MX	Premium	6	28.7
ES	Free	4	13.1
CO	Free	3	12.4
AR	Familiar	4	11.3
MX	Free	3	9.7

Ejercicio 10

Enunciado: Recursos Humanos necesita el organigrama completo. Con una CTE recursiva, muestra nombre, puesto, nivel (el CEO es 0) y ruta concatenada, partiendo desde el CEO.

```
WITH organigrama AS ( -- Miembro Ancla (Nivel 0) SELECT empleado_id, nombre,
puesto, 0 AS nivel, CAST(nombre AS nvarchar(500)) AS ruta FROM dbo.empleados WHERE
jefe_id IS NULL UNION ALL -- Miembro Recursivo SELECT e.empleado_id, e.nombre, e.puesto,
o.nivel + 1, CAST(o.ruta + N' > ' + e.nombre AS nvarchar(500)) FROM dbo.empleados AS e
JOIN organigrama AS o ON e.jefe_id = o.empleado_id ) SELECT nombre, puesto, nivel, ruta
FROM organigrama ORDER BY ruta;
```

Explicación de CTE Recursivas jerárquicas: La magia ocurre en la condición del JOIN dentro del miembro recursivo: `e.jefe\_id = o.empleado\_id`. Traducido al español, esto pide a la base de datos: "Busca en la tabla `empleados` registros cuyo `jefe\_id` coincida con el `empleado\_id` que acabo de procesar en el nivel anterior de la CTE". El motor encola las llamadas hasta que un nivel de subordinados no tiene a nadie a su cargo. Los `CAST` son obligatorios para forzar un ancho fijo en la cadena recursiva, evitando el error de discordancia de tipos.

nombre	puesto	nivel	ruta
Irene Salas	CEO	0	Irene Salas
Nuria Campos	CMO	1	Irene Salas > Nuria Campos
Laura Gil	Marketing Manager	2	Irene Salas > Nuria Campos > Laura Gil
Andrés Pozo	Community Manager	3	Irene Salas > Nuria Campos > Laura Gil > Andrés Pozo
Tomás Vidal	CTO	1	Irene Salas > Tomás Vidal
Hugo Marín	Backend Lead	2	Irene Salas > Tomás Vidal > Hugo Marín
Raúl Benet	Head of Data	2	Irene Salas > Tomás Vidal > Raúl Benet
Carmen Lozano	Data Analyst	3	Irene Salas > Tomás Vidal > Raúl Benet > Carmen Lozano
Elena Ruiz	Data Engineer	3	Irene Salas > Tomás Vidal > Raúl Benet > Elena Ruiz
Pablo Ortega	Data Engineer	3	Irene Salas > Tomás Vidal > Raúl Benet > Pablo Ortega

Ejercicio 11

Enunciado: Playlist "Todo Urbano". Obtén todas las canciones del género Urbano y sus subgéneros, a cualquier profundidad. La CTE recursiva debe partir del nombre del género, no de su id.

```
WITH arbol_urbano AS ( SELECT genero_id, nombre AS genero_concreto FROM
dbo.generos WHERE nombre = N'Urbano' UNION ALL SELECT g.genero_id, g.nombre FROM
dbo.generos AS g JOIN arbol_urbano AS a ON g.genero_padre_id = a.genero_id ) SELECT
c.titulo, a.genero_concreto AS genero, ar.nombre AS artista FROM arbol_urbano AS a JOIN
dbo.canciones AS c ON c.genero_id = a.genero_id JOIN dbo.artistas AS ar ON c.artista_id =
ar.artista_id ORDER BY a.genero_concreto, c.titulo;
```

Explicación: El ancla inicializa la búsqueda aislando el registro donde `nombre = 'Urbano'`. A partir de ahí fluye la jerarquía hacia abajo. Tras construir el árbol de IDs válidos en la memoria, se cruza esta estructura con `canciones` de forma transparente.

titulo	genero	artista
Galaxia Dembow	Dembow	MC Brisa
Rimas de Barrio	Hip Hop	Verso Libre
Calle 23	Reguetón	MC Brisa
Perreo Lunar	Reguetón	MC Brisa
Diamantes Rotos	Trap	Nébula
Humo Violeta	Trap	Nébula
Sin Cobertura	Trap	Nébula

Ejercicio 12

Enunciado: El CTO quiere saber quién forma parte de su área, directa o indirectamente. Con una CTE recursiva partiendo de Tomás Vidal, muestra nombre, puesto y nivel relativo, omitiendo al propio CTO.

```
WITH equipo_cto AS ( SELECT empleado_id, nombre, puesto, 0 AS nivel FROM
dbo.empleados WHERE nombre = N'Tomás Vidal' UNION ALL SELECT e.empleado_id, e.nombre,
e.puesto, c.nivel + 1 FROM dbo.empleados AS e JOIN equipo_cto AS c ON e.jefe_id =
c.empleado_id ) SELECT nombre, puesto, nivel FROM equipo_cto WHERE nivel > 0 ORDER BY
nivel, nombre;
```

Explicación: Este patrón es útil para crear sub-grafos de jerarquías sin procesar la tabla entera. El filtro `WHERE nivel > 0` en la consulta final excluye al ancla (el CTO) manteniendo intacto su uso para iniciar la recursión interna.

nombre	puesto	nivel
Hugo Marín	Backend Lead	1
Raúl Benet	Head of Data	1
Carmen Lozano	Data Analyst	2
Elena Ruiz	Data Engineer	2
Pablo Ortega	Data Engineer	2